I. LEARNING OBJECTIVES

At the end of this activity, students should be able to:

1. Explain the role of middleware in an Express.js application.
2. Create a dedicated folder for middleware components.
3. Create custom middleware as separate JavaScript modules.
4. Export and import middleware functions using `module.exports` and `require()`.
5. Integrate middleware into an existing Express.js project.
6. Implement middleware chaining using `next()`.
7. Implement basic authentication and authorization middleware.
8. Implement centralized error-handling middleware.
9. Test the middleware using the existing API routes.

II. IMPORTANT INSTRUCTION

DO NOT CREATE A NEW EXPRESS PROJECT.

For this activity, you will use the **same Express.js project that you completed in the previous activity on HTTP Methods.**

Your previous project already contains your:

- `GET` routes
- `POST` routes
- `PUT` routes
- `DELETE` routes
- request and response handling
- API testing using Postman, Thunder Client, or `test.http`

You will **not replace your existing routes**.

Instead, you will **integrate middleware into your existing project.**

III. FROM HTTP METHODS TO MIDDLEWARE

In the previous activity, your application probably followed this basic flow:

```
Client Request
      ↓
 HTTP Method
      ↓
    Route
      ↓
  Response
```

In this activity, we will add middleware:

```
Client Request
      ↓
  Middleware
      ↓
HTTP Method / Route
      ↓
  Response
```

When several middleware functions are used:

```
Client Request
      ↓
 Middleware 1
      ↓
 Middleware 2
      ↓
 Middleware 3
      ↓
Route Handler
```

```
↓
Response
```

Middleware functions can intercept requests and responses before they reach the actual route handlers. They can be used for tasks such as logging, authentication, and error handling.

IV. PRE-ACTIVITY CHECK

Before proceeding, open your **previous Express.js project**.

Check if your project still works.

Run your application using your existing command.

For example:

```
node app.js
```

or:

```
npm start
```

Test at least one of your existing routes.

Example:

```
GET http://localhost:3000/students
```

Checkpoint 1

Does your previous project still work?

☐ Yes

☐ No

If **No**, fix your previous project before proceeding.

V. ACTIVITY 1 – CREATE THE MIDDLEWARE FOLDER

We will now organize our middleware separately from the main application.

Inside your **existing project**, create a folder named:

```
middleware
```

Your project should now look similar to:

```
your-project/
│
├── node_modules/
│
├── middleware/
│
├── app.js
│
├── package.json
└── package-lock.json
```

Important: All custom middleware that you will create in this activity must be placed inside the `middleware` folder.

VI. ACTIVITY 2 – CREATE THE LOGGER MIDDLEWARE

The first middleware will be responsible for displaying information about incoming requests.

Inside the `middleware` folder, create:

```
logger.js
```

Your structure should now be:

```
your-project/
│
├── middleware/
│   └── logger.js
│
├── app.js
└── package.json
```

Step 1 – Write the Middleware

Inside `logger.js`, write:

```
function logRequests(req, res, next) {
```

```
    console.log(`${req.method} ${req.url}`);

    next();
}

module.exports = logRequests;
```

Step 2 – Understand the Code
Look at:
`function logRequests(req, res, next)`
The three parameters are:

Parameter	Purpose
`req`	Contains information about the incoming request
`res`	Represents the response
`next`	Passes control to the next middleware or route

A standard middleware function uses `req`, `res`, and `next`, and `next()` allows the request-response cycle to continue.

Step 3 – Export the Middleware
At the bottom:
`module.exports = logRequests;`
This allows another JavaScript file to use the middleware.

VII. ACTIVITY 3 – IMPORT THE LOGGER INTO app.js

Open your existing `app.js`.
At the top of the file, add:
`const logRequests = require('./middleware/logger');`
You are now importing the middleware from:
`middleware/logger.js`

Step 2 – Register the Middleware
After:
`const app = express();`
add:
`app.use(logRequests);`
Your structure should look similar to:

```
const express = require('express');

const app = express();

const logRequests = require('./middleware/logger');

app.use(logRequests);

// Your existing routes below
```

VIII. ACTIVITY 4 – TEST THE LOGGER

Run your existing application.
Test your existing routes.
For example:

```
GET /students
POST /students
PUT /students/1
```

```
DELETE /students/1
```

Check the terminal.
You should see:

```
GET /students
POST /students
PUT /students/1
DELETE /students/1
```

The module describes this type of custom logging middleware as a way to capture incoming traffic before routing takes place.

Guide Question 1
Why does `logRequests` execute when you access different routes even though you only wrote:
`app.use(logRequests);`
Answer:

IX. ACTIVITY 5 – IMPROVE THE LOGGER

Modify `middleware/logger.js`.
Change your middleware to:

```
function logRequests(req, res, next) {
    console.log('--- INCOMING REQUEST ---');
    console.log('Method:', req.method);
    console.log('URL:', req.url);
    console.log('Time:', new Date().toLocaleString());

    next();
}

module.exports = logRequests;
```

Test your existing routes again.
Expected Output

```
--- INCOMING REQUEST ---
Method: GET
URL: /students
Time: 9/13/2026, 8:30:00 PM
```

X. ACTIVITY 6 – INTEGRATE BUILT-IN MIDDLEWARE

Not all middleware needs to be created manually.
Express provides built-in middleware.
For this activity, use:
`express.json()`
The module identifies `express.json()` as a built-in middleware for processing incoming JSON data.
In your existing `app.js`, add:
`app.use(express.json());`
Your middleware section should now look similar to:

```
const express = require('express');

const app = express();

const logRequests = require('./middleware/logger');
```

```
app.use(express.json());
app.use(logRequests);
```

XI. ACTIVITY 7 – TEST YOUR EXISTING POST ROUTE

Do **not create another POST route**.
Use the POST route you created during the HTTP Methods activity.
For example:

```
POST http://localhost:3000/students
Content-Type: application/json
```

Request body:

```
{
    "name": "Juan Dela Cruz",
    "course": "BSIT",
    "year": 1
}
```

Check whether your existing POST route still works.

Guide Question 2
Why is `express.json()` important when receiving JSON data from the client?
Answer:

XII. ACTIVITY 8 – CREATE AUTHENTICATION MIDDLEWARE

We will now create another middleware.
Inside the same `middleware` folder, create:

```
auth.js
```

Your folder should now be:

```
middleware/
│
├── logger.js
└── auth.js
```

Step 1 – Create the Middleware
Inside `auth.js`:

```
function authenticateUser(req, res, next) {
    const token = req.headers.authorization;

    if (!token) {
        return res.status(401).json({
            message: 'Unauthorized access'
        });
    }

    next();
}

module.exports = authenticateUser;
```

The module presents authentication middleware as a gatekeeper that checks incoming request headers before allowing access to restricted routes.

XIII. ACTIVITY 9 – IMPORT AUTHENTICATION MIDDLEWARE

In `app.js`, add:

```
const authenticateUser = require('./middleware/auth');
```

You now have:

```
const logRequests = require('./middleware/logger');
const authenticateUser = require('./middleware/auth');
```

XIV. ACTIVITY 10 – PROTECT AN EXISTING ROUTE

Choose **one existing route** from your project that you want to protect.
For example, if you currently have:

```
app.get('/students', (req, res) => {
    res.json(students);
});
```

Modify it to:

```
app.get('/students', authenticateUser, (req, res) => {
    res.json(students);
});
```

Notice that we did not create a new route.
We simply integrated middleware into an **existing route**.

XV. ACTIVITY 11 – TEST AUTHENTICATION

Test 1 – Without Authorization

Send:

```
GET http://localhost:3000/students
```

Expected:

```
401 Unauthorized
```

Test 2 – With Authorization

Add the following header:

```
Authorization: Bearer 12345
```

Send the request again.
Expected:

```
200 OK
```

Your existing route response should now appear.

XVI. ACTIVITY 12 – CREATE ROLE MIDDLEWARE

Inside the same `middleware` folder, create:

```
role.js
```

Your folder should now look like:

```
middleware/
│
├── logger.js
├── auth.js
└── role.js
```

Inside `role.js`:

```
function checkRole(req, res, next) {
    const role = req.headers['x-user-role'];

    if (role !== 'Faculty') {
        return res.status(403).json({
            message: 'Faculty access only'
        });
    }
```

```
    next();
}

module.exports = checkRole;
```

XVII. ACTIVITY 13 – IMPORT ROLE MIDDLEWARE

In `app.js`:

```
const checkRole = require('./middleware/role');
```

You should now have:

```
const logRequests = require('./middleware/logger');
const authenticateUser = require('./middleware/auth');
const checkRole = require('./middleware/role');
```

XVIII. ACTIVITY 14 – CREATE A MIDDLEWARE CHAIN

Go back to the same existing protected route.

Change:

```
app.get('/students', authenticateUser, (req, res) => {
    res.json(students);
});
```

into:

```
app.get(
    '/students',
    authenticateUser,
    checkRole,
    (req, res) => {
        res.json(students);
    }
);
```

Now two middleware functions are executed before the route handler.

XIX. UNDERSTAND THE CHAIN

The request now follows:

```
Client Request
      ↓
logRequests
      ↓
authenticateUser
      ↓
checkRole
      ↓
Existing Route Handler
      ↓
Response
```

Each middleware calls:

```
next();
```

when it allows the request to continue.

The module emphasizes that `next()` passes control to the subsequent function in the middleware execution sequence.

XX. ACTIVITY 15 – TEST THE MIDDLEWARE CHAIN

Test the same existing route three times.

TEST 1 – NO HEADERS

```
GET http://localhost:3000/students
```
Expected:
```
401 Unauthorized
```

TEST 2 – WRONG ROLE
Headers:
```
Authorization: Bearer 12345
x-user-role: Student
```
Expected:
```
403 Forbidden
```

TEST 3 – CORRECT ROLE
Headers:
```
Authorization: Bearer 12345
x-user-role: Faculty
```
Expected:
```
200 OK
```

XXI. ACTIVITY 16 – CREATE ERROR-HANDLING MIDDLEWARE

Now we will create the final middleware.
Inside the same folder:
`middleware/errorHandler.js`
Your folder should now contain:
```
middleware/
│
├── logger.js
├── auth.js
├── role.js
└── errorHandler.js
```
Inside `errorHandler.js`:
```
function errorHandler(err, req, res, next) {
    console.error(err.stack);

    res.status(500).json({
        message: 'Something went wrong!',
        error: err.message
    });
}

module.exports = errorHandler;
```
Error-handling middleware is identified by its four parameters:
`err, req, res, next`
The module specifically uses this four-parameter structure for error-handling middleware.

XXII. ACTIVITY 17 – IMPORT THE ERROR HANDLER

In `app.js`:
```
const errorHandler = require('./middleware/errorHandler');
```

XXIII. ACTIVITY 18 – REGISTER THE ERROR HANDLER

Place the error handler **after your existing routes**:
```
app.use(errorHandler);
```
Your overall structure should now be similar to:
```
const express = require('express');

const app = express();
```

```
// Middleware imports
const logRequests = require('./middleware/logger');
const authenticateUser = require('./middleware/auth');
const checkRole = require('./middleware/role');
const errorHandler = require('./middleware/errorHandler');

// Built-in middleware
app.use(express.json());

// Global middleware
app.use(logRequests);


// EXISTING ROUTES

app.get('/students', authenticateUser, checkRole, (req, res) => {
    res.json(students);
});

app.post('/students', (req, res) => {
    // Your existing POST code
});

app.put('/students/:id', (req, res) => {
    // Your existing PUT code
});

app.delete('/students/:id', (req, res) => {
    // Your existing DELETE code
});


// Error-handling middleware
app.use(errorHandler);
```

Do not replace your existing route logic with the comments above.
Keep your original route code.

XXIV. ACTIVITY 19 – TEST ERROR HANDLING

Add a temporary test route:

```
app.get('/test-error', (req, res, next) => {
    next(new Error('Test Error'));
});
```

Then access:

```
GET /test-error
```

Expected response:

```
{
    "message": "Something went wrong!",
    "error": "Test Error"
}
```

The module demonstrates passing an error to the centralized error handler through `next(err)`.

XXV. FINAL PROJECT STRUCTURE

At the end of this activity, your existing project should look similar to:

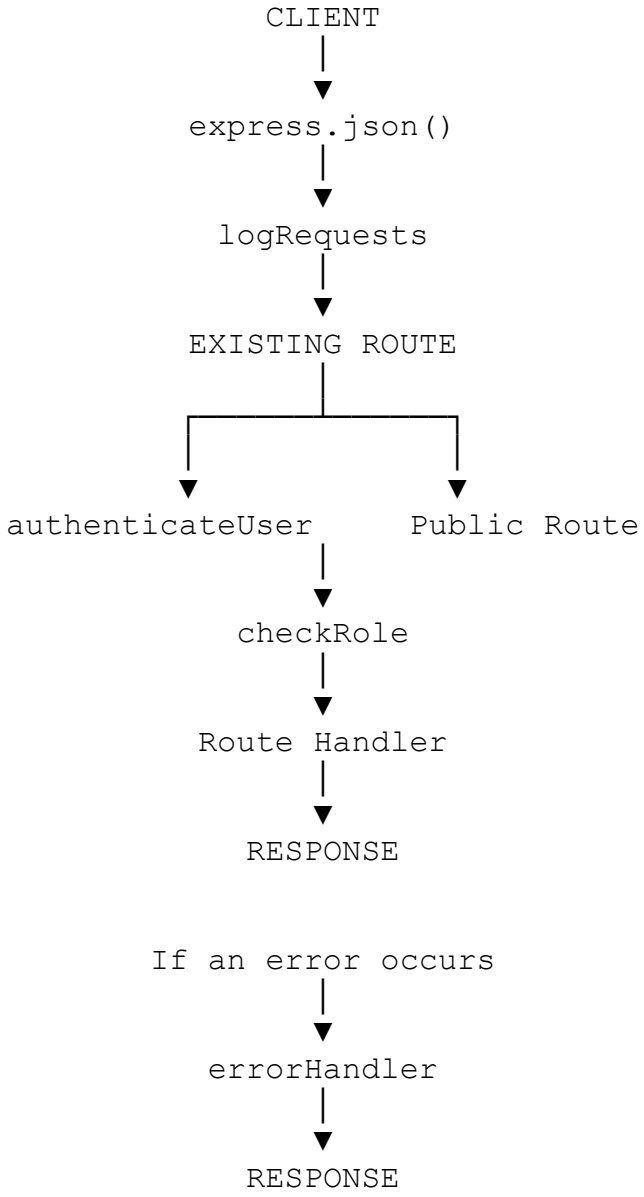
```
your-existing-project/
```

```
├── node_modules/
│
├── middleware/
│   ├── logger.js
│   ├── auth.js
│   ├── role.js
│   └── errorHandler.js
│
├── app.js
│
├── package.json
└── package-lock.json
```

The important point is that your middleware is now separated from your main application logic.

XXVI. FINAL MIDDLEWARE FLOW

Your project should now demonstrate the following:



XXVII. TESTING CHECKLIST

After completing the integration, test your **original project**.

Requirement	Expected Result	Completed
Existing GET route	Still works	☐

Existing POST route	Receives JSON	☐
Existing PUT route	Still works	☐
Existing DELETE route	Still works	☐
`logger.js`	Logs requests	☐
`auth.js`	Checks authorization	☐
`role.js`	Checks user role	☐
Middleware chaining	Uses `next()`	☐
No authorization	Returns 401	☐
Wrong role	Returns 403	☐
Correct role	Returns 200	☐
`errorHandler.js`	Handles errors	☐

XXVIII. GUIDE QUESTIONS

1. Why did we create a separate `middleware` folder?

2. What is the purpose of `module.exports`?

3. What is the purpose of `require('./middleware/logger')`?

4. What is the purpose of `next()`?

5. What happens if a middleware neither sends a response nor calls `next()`?

6. What is the difference between authentication and authorization?

7. What is the difference between HTTP status `401` and `403`?

8. Why should the error-handling middleware be registered after the routes?

XXIX. DEBUGGING CHALLENGE

A student created the following middleware:

```
function logRequests(req, res, next) {
    console.log(req.method, req.url);
}

module.exports = logRequests;
```

The existing routes no longer respond and the browser keeps loading.

Question:

What is missing from the middleware?

Correct the code:

```
function logRequests(req, res, next) {
    console.log(req.method, req.url);

    _________________________;
}

module.exports = logRequests;
```

XXX. FINAL INTEGRATION TASK

Enhance your **existing HTTP Methods project**.

You must integrate the following:

Folder

```
middleware/
```

Middleware Files

```
logger.js
auth.js
role.js
errorHandler.js
```

Your Existing Application

Keep your original:

```
GET
POST
PUT
DELETE
```

routes.

You are only adding middleware to improve the application.

XXXI. REQUIRED DEMONSTRATION

During submission or demonstration, show the following:

1. Project Structure

Show:

```
middleware/
├── logger.js
├── auth.js
├── role.js
└── errorHandler.js
```

2. Existing HTTP Methods

Demonstrate that your previous:

- GET
- POST
- PUT
- DELETE

operations still work.

3. Logging

Show the terminal output generated by `logger.js`.

4. Authentication

Show:

`401 Unauthorized`

when no authorization header is provided.

5. Authorization

Show:

`403 Forbidden`

when the user has the wrong role.

6. Successful Access

Show:

`200 OK`

when the correct authorization and role are provided.

7. Error Handling

Show the response generated by:

`errorHandler.js`

XXXII. SUBMISSION REQUIREMENTS

Submit your **updated existing Express.js project**.

Required:

- ☐ Existing HTTP Methods are still functional.
- ☐ `middleware` folder is created.
- ☐ `logger.js` is created.
- ☐ `auth.js` is created.
- ☐ `role.js` is created.
- ☐ `errorHandler.js` is created.
- ☐ Middleware functions are exported.
- ☐ Middleware functions are imported into `app.js`.
- ☐ `express.json()` is integrated.
- ☐ `next()` is properly used.
- ☐ Middleware chaining is demonstrated.
- ☐ Authentication is demonstrated.
- ☐ Authorization is demonstrated.
- ☐ Error handling is demonstrated.

XXXIII. REFLECTION

Answer in **5–7 sentences**:

How does integrating middleware improve the organization and functionality of your existing Express.js project? **Reflection per student**.

Student Name	Reflection
1.	
2.	